

Plugin Database Schema

JSON Data Model for 100+ Supported Plugins

- **JSON Data Model for 100+ Supported Plugins**

Category: Data Infrastructure — Plugin Database

Importance: ★★★★★ — Core system dependency

- **1. Plugin Database Overview**

The plugin database is the **central registry** of all plugins supported by WhyCremisi. It is a single `plugin-s.json` file managed under git version control, containing the complete definition of every plugin: parameters, presets, recommended procedures, and metadata.

◆ TEST PIPELINE

PLUGIN DATABASE

plugins.json (version 2.0, ~5MB for 100 plugins)

pluginEntry[]

metadata (vendor, version, formats)

parameters[] (up to 500 per plugin)

presets[] (up to 300 factory presets)

knownProcedures[] (AI configurations)

mappings[] (VST3 GUID, MIDI CC, OSC)

Git: plugins.json → versioned, mandatory PR review

CI: JSON Schema validation + integrity tests

```
{
  "schemaVersion": "2.0",
  "lastUpdated": "2026-05-12",
  "pluginCount": 100,
  "plugins": [
    { /* PluginEntry */ },
    { /* PluginEntry */ }
  ],
  "categories": { /* CategoryDefinition */ },
  "$schema": "../schema/plugin-database.schema.json"
}
```

• 2. Core Schema — PluginEntry

Every plugin is represented by a `PluginEntry` object with the following structure:

```
{
  "id": "fabfilter_proq3",
  "name": "Pro-Q 3",
  "vendor": "FabFilter",
  "version": "3.22",
  "category": "ParametricEQ",
  "subcategory": "DynamicEQ",
  "formats": ["VST3", "AU", "AAX"],
  "vst3Guid": "E8E38A93-21B0-4B8D-B7EC-B54B8B9CF48C",
  "auComponentId": "FFEQ",
  "aaxId": "com.fabfilter.ProQ3",
  "parameters": [ /* ParameterEntry[] */ ],
  "presets": [ /* PresetEntry[] */ ],
  "knownProcedures": [ /* ProcedureEntry[] */ ],
  "metadata": {
    "tags": ["eq", "dynamic-eq", "linear-phase", "spectrum"],
    "imageUrl": "https://cdn.fabfilter.com/img/pro-q3.png",
    "manualUrl": "https://www.fabfilter.com/help/pro-q3",
    "minVersion": "3.0",
    "testedVersion": "3.22",
    "releaseDate": "2024-09-15",
    "categoryPath": ["EQ", "Parametric", "Dynamic"]
  }
}
```

FIELD	TYPE	DESCRIPTION	REQUIRED
<code>id</code>	string	Unique snake_case identifier	✓
<code>name</code>	string	Commercial plugin name	✓
<code>vendor</code>	string	Developer / company	✓
<code>version</code>	string	Tested version	✓
<code>category</code>	string	Primary category (see §4)	✓
<code>formats</code>	string[]	Supported formats	✓
<code>parameters</code>	array	Exposed parameter list	✓
<code>metadata.tags</code>	string[]	Semantic search tags	✗
<code>metadata.manualUrl</code>	string	User manual URL	✗
<code>metadata.minVersion</code>	string	Minimum required version	✗

[NOTE] The ``vst3Guid`` field is critical for automatic plugin recognition when loaded in a VST3 host. Without a correct GUID, WhyCremisi cannot deterministically identify the plugin.

• 3. Parameter Schema — ParameterEntry

The heart of the database: every parameter of every plugin is defined with maximum precision.

— 3.1 COMPLETE STRUCTURE

```
{
  "id": "band1_frequency",
  "name": "Band 1 Frequency",
  "index": 0,
  "type": "float",
  "defaultValue": 1000.0,
  "min": 20.0,
  "max": 20000.0,
  "step": 0.1,
  "unit": "Hz",
  "group": "Band 1",
  "hidden": false,
  "automationRate": "continuous",
  "mapping": {
    "midiCC": 74,
    "oscPath": "/plugin/fabfilter/proq3/band1/freq"
  },
  "aliases": ["freq1", "frequency band 1"],
  "scaling": {
    "type": "logarithmic",
    "normalizedMin": 0.0,
    "normalizedMax": 1.0,
    "physicalMin": 20.0,
    "physicalMax": 20000.0
  }
}
```

— 3.2 PARAMETER TYPES

◆ OVERVIEW

TYPE | EXAMPLES | RANGE

float | Gain, Frequency | min..max (continuous)

int | Bands, Oversampling | min..max (discrete)

bool | Bypass, Solo, Mute | 0 / 1

enum | Filter Type, Style | string list

text | Preset Name | free-form string

— 3.3 PARAMETER GROUPS

Modern plugins often organize parameters into logical sections:

```
{
  "groups": {
    "Band 1": { "index": 0, "params": ["band1_frequency", "band1_gain", "band1_q", "band1_type", "band1_automationRate"], "hidden": false },
    "Band 2": { "index": 1, "params": ["band2_frequency", "band2_gain", "band2_q", "band2_type", "band2_automationRate"], "hidden": false },
    "Output": { "index": 24, "params": ["output_gain", "phase_mode", "bypass"] }
  }
}
```

• 4. Category Schema

Plugin taxonomy uses a flat hierarchy with cross-cutting tags for flexibility.

— 4.1 MAIN CATEGORIES

◆ PROCESS FLOW

EQ

GraphicEQ

ParametricEQ

DynamicEQ

Shelf

FilterBank

Crossover

Dynamics

Compressor

TubeComp

BusComp

MasteringComp

MultibandComp

Limiter

SoftClip

BrickwallLimiter

TruePeakLimiter

Gate

— 4.2 CATEGORY MAP IN JSON

```
{
  "categories": {
    "ParametricEQ": {
      "displayName": "Parametric EQ",
      "icon": "eq-parametric",
      "parents": ["EQ"],
      "color": "#4FC3F7"
    },
    "BrickwallLimiter": {
      "displayName": "Brickwall Limiter",
      "icon": "limiter-brickwall",
      "parents": ["Dynamics", "Limiter"],
      "color": "#FF5252"
    },
    "TubeComp": {
      "displayName": "Tube Compressor",
      "icon": "comp-tube",
      "parents": ["Dynamics", "Compressor"],
      "color": "#FFB74D"
    }
  }
}
```

◆ PROCESS FLOW

CATEGORY	VST3	AU	AAX	AAX-S	Notes
ParametricEQ	✓	✓	✓	✓	
DynamicEQ	✓	✓	✓	✓	
Compressor	✓	✓	✓	✓	
MultibandComp	✓	✓	✓	✗	AAX-S unsupported
Limiter	✓	✓	✓	✓	
Reverb	✓	✓	✓	✓	
Synth	✓	✓	✗	✗	AAX no instruments
Sampler	✓	✓	✗	✗	
SpectralProcessor	✓	✓	✓	✗	

• 5. Factory Presets

Every plugin can have predefined factory presets that WhyCremisi uses as starting points for AI-driven processing.

— 5.1 PRESET STRUCTURE

```
{
  "id": "proq3_vocal_clarity",
  "name": "Vocal Clarity",
  "pluginId": "fabfilter_proq3",
  "category": "vocals",
  "params": {
    "band1_type": "HighPass",
    "band1_frequency": 80,
    "band1_q": 0.707,
    "band2_type": "Peak",
    "band2_frequency": 320,
    "band2_gain": -2.5,
    "band2_q": 1.2,
    "band3_type": "Peak",
    "band3_frequency": 3500,
    "band3_gain": 2.0,
    "band3_q": 0.8,
    "output_gain": 0.0
  },
  "tags": ["vocals", "clarity", "presence", "clean"],
  "rating": 4.5,
  "author": "WhyCremisi Factory",
  "description": "High-pass at 80Hz, subtle cut at 320Hz for mud, 3.5kHz presence boost"
}
```

◆ PLUGIN CONTROL FLOW



[NOTE] Factory presets serve as the AI's starting point. WhyCremisi uses them as a baseline and modifies them dynamically based on session context, user preferences, and FFT analysis of the audio material.

• 6. Known Procedures

Known procedures are sequences of operations that WhyCremisi can execute automatically across one or more plugins. They represent the system's codified "know-how."

```
{
  "id": "vocal_chain_setup",
  "name": "Configure Vocal Chain",
  "description": "Sets up a complete vocal processing chain with EQ, compression, and de-essing",
  "category": "mixing",
  "tags": ["vocals", "chain", "mixing"],
  "steps": [
    {
      "action": "setParameter",
      "target": "plugin",
      "pluginId": "fabfilter_proq3",
      "params": {
        "band1_type": "HighPass",
        "band1_frequency": 80,
        "band1_enabled": true
      },
      "description": "High-pass filter at 80Hz"
    },
    {
      "action": "setParameter",
      "target": "plugin",
      "pluginId": "fabfilter_proc2",
      "params": {
        "style": "Vocal",
        "threshold": -18.0,
        "ratio": 3.0,
        "attack": 0.5,
        "release": 50.0,
        "makeup_gain": 2.0
      },
      "description": "Set compressor for vocals"
    },
    {
      "action": "setParameter",
      "target": "plugin",
      "pluginId": "fabfilter_prods2",
      "params": {
        "frequency": 5000,
        "amount": 3.0,
        "mode": "Split"
      },
      "description": "De-esser at 5kHz"
    },
    {
      "action": "analyze",
      "target": "fft",
      "params": {
        "windowSize": 4096,
        "checkRange": [80, 20000]
      },
      "description": "Verify spectral balance after chain setup"
    }
  ],
  "pluginIds": ["fabfilter_proq3", "fabfilter_proc2", "fabfilter_prods2"],
  "estimatedTimeMs": 45
}
```


6.2 DEFAULT PROCEDURES (PARTIAL)

PROCEDURE	PLUGINS INVOLVED	ACTIONS
Vocal Chain Setup	Pro-Q3, Pro-C2, Pro-DS2	HPF → Compression → De-ess
Master Bus Chain	Pro-MB, Pro-L2	Multiband → Limiter
Sidechain Setup	Pro-C2 + kick	Key input → Threshold → Ratio
Parallel Compression	Pro-C2 (30% mix)	Blend wet/dry
Bass Cleanup	Pro-Q3	HPF 40Hz → Cut mud 200-400Hz
Room Taming	ValhallaRoom + Pro-Q3	Reverb → EQ cut sibilance
Stereo Widening	Imager	M/S → Width expansion

6.3 PROCEDURE EXECUTION FLOW

◆ OVERVIEW

User: "set up my vocal chain"

WhyCremisi lookup → "vocal_chain_setup"

Step 1: Pro-Q3 → HPF @ 80Hz

Step 2: Pro-C2 → Threshold -18dB, Ratio 3:1

Step 3: Pro-DS2 → De-ess @ 5kHz

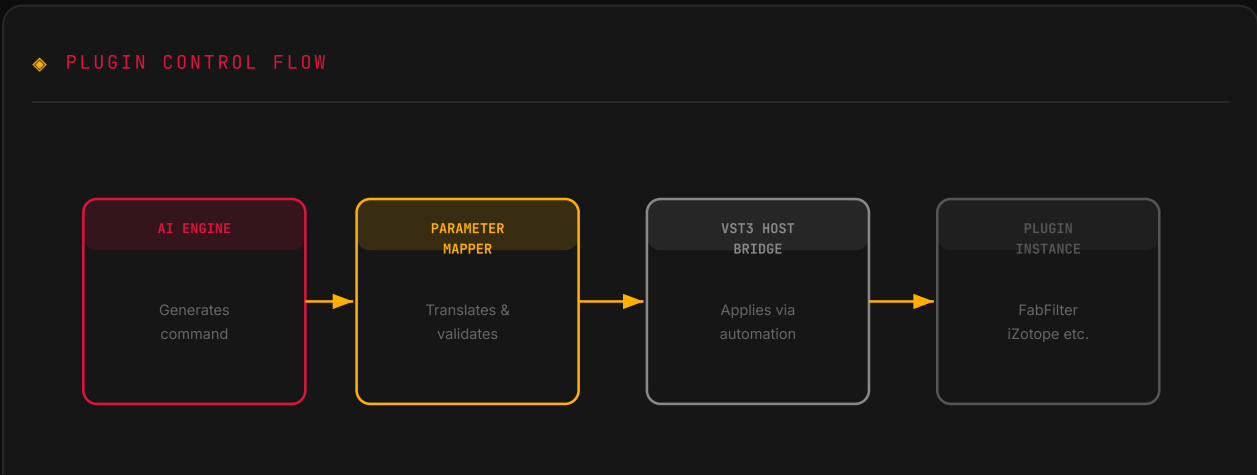
Step 4: Verify → FFT check

[Propose to user with undo available]

7. DAW ↔ Plugin Mapping

Plugin recognition is the mechanism by which WhyCremisi identifies which plugin is loaded on which track.

7.1 THREE-LEVEL RECOGNITION STRATEGY



— 7.2 MAPPING STRUCTURE

```
{
  "mappings": {
    "vst3Guid": { "type": "exact", "field": "vst3Guid" },
    "auComponentId": { "type": "exact", "field": "auComponentId" },
    "aaxId": { "type": "exact", "field": "aaxId" },
    "vendorFuzzy": {
      "type": "fuzzy",
      "fields": ["vendor", "name"],
      "threshold": 0.85
    },
    "parameterSignature": {
      "type": "ml",
      "description": "ML-based matching on parameter structure",
      "status": "planned"
    }
  }
}
```

— 7.3 HIDDEN PARAMETERS AND DISCOVERY

[NOTE] Some plugins expose internal or hidden parameters only through explicit scanning. WhyCremisi performs a full parameter scan on first connection and compares against the database. Undocumented parameters are flagged with "discovered": true for manual review.

```
{
  "id": "unmapped_param_47",
  "name": "Internal Param 47",
  "index": 47,
  "type": "float",
  "discovered": true,
  "needsReview": true,
  "notes": "Discovered during automatic scan – undocumented parameter not found in official manual"
}
```

— 8.1 FABFILTER PRO-Q 3 — 24 BANDS, 5 FILTER TYPES, SPECTRUM

```
{
  "id": "fabfilter_proq3",
  "name": "Pro-Q 3",
  "vendor": "FabFilter",
  "category": "ParametricEQ",
  "parameters": [
    { "id": "band1_freq", "index": 0, "type": "float", "min": 20, "max": 20000, "unit": "Hz", "default": 1000 },
    { "id": "band1_gain", "index": 1, "type": "float", "min": -30, "max": 30, "unit": "dB", "defaultValue": 0 },
    { "id": "band1_q", "index": 2, "type": "float", "min": 0.025, "max": 40, "defaultValue": 0.707 },
    { "id": "band1_type", "index": 3, "type": "enum", "values": ["Peak", "LowShelf", "HighShelf", "LowPass", "HighPass"], "defaultValue": "Peak" },
    { "id": "band1_enabled", "index": 4, "type": "bool", "defaultValue": true }
  ],
  "parameterCount": 200,
  "presetCount": 25,
  "knownProcedures": ["vocal_cleanup", "bass_tighten", "mastering_eq"]
}
```

— 8.2 IZOTOPE OZONE 10 — 8 MODULES, MASTER ASSISTANT

```
{
  "id": "izotope_ozone10",
  "name": "Ozone 10",
  "vendor": "iZotope",
  "category": "LoudnessMaximizer",
  "modules": [
    { "name": "EQ", "enabled": true, "parameters": ["eq_band1_freq", "eq_band1_gain", "eq_band1_q"] },
    { "name": "Dynamics", "enabled": true, "parameters": ["dynamics_threshold", "dynamics_ratio", "dynamics_attack", "dynamics_release"] },
    { "name": "Imager", "enabled": true, "parameters": ["imager_width", "imager_stereo_balance"] },
    { "name": "Maximizer", "enabled": true, "parameters": ["maximizer_threshold", "maximizer_irc_mode"] },
    { "name": "Match EQ", "enabled": false },
    { "name": "Spectral Shaper", "enabled": false },
    { "name": "Vintage Tape", "enabled": false },
    { "name": "LowEnd Focus", "enabled": false }
  ],
  "masterAssistant": {
    "available": true,
    "params": ["target_loudness", "target_dynamics", "style"]
  },
  "parameterCount": 180
}
```

```
{
  "id": "valhalla_valhallaroom",
  "name": "ValhallaRoom",
  "vendor": "ValhallaDSP",
  "category": "Reverb",
  "algorithms": ["Nano", "Warm", "Smooth", "Quiet", "Bright", "Concrete", "Chamber", "Hall", "Plate",
  "parameters": [
    { "id": "mix", "index": 0, "type": "float", "min": 0, "max": 100, "unit": "%" },
    { "id": "decay", "index": 1, "type": "float", "min": 0.01, "max": 100.0, "unit": "s" },
    { "id": "pre_delay", "index": 2, "type": "float", "min": 0, "max": 500, "unit": "ms" },
    { "id": "size", "index": 3, "type": "float", "min": 0.01, "max": 4.0 },
    { "id": "diffusion", "index": 4, "type": "float", "min": 0, "max": 100 },
    { "id": "modulation_rate", "index": 5, "type": "float", "min": 0, "max": 20, "unit": "Hz" },
    { "id": "modulation_depth", "index": 6, "type": "float", "min": 0, "max": 100 },
    { "id": "high_cut", "index": 7, "type": "float", "min": 100, "max": 20000, "unit": "Hz" },
    { "id": "low_cut", "index": 8, "type": "float", "min": 20, "max": 2000, "unit": "Hz" },
    { "id": "algorithm", "index": 9, "type": "enum", "values": ["Nano", "Warm", "Smooth", "Quiet", "Br
  ],
  "parameterCount": 40,
  "presetCount": 128
}
```

— 8.4 XFER SERUM — WAVETABLE OSCILLATOR, 45 PARAMETERS, 200+ PRESETS

```
{
  "id": "xfer_serum",
  "name": "Serum",
  "vendor": "Xfer Records",
  "category": "Synth",
  "oscillators": [
    { "id": "oscA", "type": "wavetable", "parameters": ["osc_a_wavetable", "osc_a_coarse", "osc_a_fine"
    { "id": "oscB", "type": "wavetable", "parameters": ["osc_b_wavetable", "osc_b_coarse", "osc_b_fine"
    { "id": "sub", "type": "sub", "parameters": ["sub_level", "sub_waveform"] },
    { "id": "noise", "type": "noise", "parameters": ["noise_level", "noise_sample"] }
  ],
  "filters": [
    { "id": "filter1", "type": "multimode", "parameters": ["filter1_type", "filter1_cutoff", "filter1
  ],
  "modulation": {
    "envelopes": 4,
    "lfo": 4,
    "matrixSlots": 16
  },
  "parameterCount": 200,
  "presetCount": 450
}
```

— 9.1 PLUGIN ADDITION WORKFLOW

◆ TEST PIPELINE

USER > GENERATE > VALIDATE > MERGE

Request JSON stub schema to main

→ →

PluginScanner tool ajv validate plugins.json

Scan VST3/AU params + integrity test

Generate initial + parameter test

JSON entry + preset test

— 9.2 COMMUNITY CONTRIBUTION MODEL

```
{
  "contributionModel": {
    "type": "crowd-sourced",
    "platform": "GitHub PR",
    "requirements": [
      "JSON Schema validation passed",
      "Plugin owned and tested by contributor",
      "At least 80% of parameters mapped",
      "Minimum 5 working presets"
    ],
    "reviewProcess": [
      "Automatic: CI validation + integrity tests",
      "Manual: parameter verification by maintainer",
      "Beta: plugin marked 'community' for 30 days",
      "Stable: promoted after user confirmation"
    ],
    "scoreSystem": {
      "baseContribution": 10,
      "perParameter": 1,
      "perPreset": 3,
      "perProcedure": 5,
      "verificationBonus": 20
    }
  }
}
```

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "PluginDatabase",
  "type": "object",
  "required": ["schemaVersion", "plugins"],
  "properties": {
    "schemaVersion": { "type": "string", "pattern": "^\\d+\\.\\.\\d+$" },
    "plugins": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "#/definitions/PluginEntry" }
    }
  },
  "definitions": {
    "PluginEntry": {
      "type": "object",
      "required": ["id", "name", "vendor", "category", "parameters"],
      "properties": {
        "id": { "type": "string", "pattern": "^[a-z][a-z0-9_]+$" },
        "name": { "type": "string" },
        "vendor": { "type": "string" },
        "category": { "type": "string", "enum": ["ParametricEQ", "DynamicEQ", "Compressor", "Limiter"] },
        "parameters": {
          "type": "array",
          "items": { "$ref": "#/definitions/ParameterEntry" }
        }
      }
    },
    "ParameterEntry": {
      "type": "object",
      "required": ["id", "name", "index", "type"],
      "properties": {
        "id": { "type": "string" },
        "name": { "type": "string" },
        "index": { "type": "integer", "minimum": 0 },
        "type": { "type": "string", "enum": ["float", "int", "bool", "enum", "text"] },
        "min": { "type": "number" },
        "max": { "type": "number" },
        "defaultValue": {}
      }
    }
  }
}

```

TEST	DESCRIPTION	COMMAND
Valid JSON	Correct JSON parsing	<code>ajv validate -s schema.json -d plugins.json</code>
Unique GUIDs	No duplicate GUIDs	<code>`jq '[.plugins[]].vst3Guid]</code>
Reachable Parameters	Contiguous indices 0..n	Custom Python script
Validated Presets	Preset params exist in plugin	<code>node tests/validate-presets.js</code>
Valid Categories	Category exists in taxonomy	<code>`jq '.plugins[].category]</code>

[NOTE] The database is automatically regenerated from scanning every time a new plugin is detected on the user's system. The scan produces a "JSON stub" that the contributor can complete with presets, procedures, and metadata.

• **Appendix A: File Structure Summary**

◆ DATA SCHEMA

`plugins.json`

```
schemaVersion: "2.0"
lastUpdated: "2026-05-12"
pluginCount: 100
plugins: PluginEntry[100]
[0]: id, name, vendor, version, category, formats
parameters: ParameterEntry[200] max
presets: PresetEntry[300] max
knownProcedures: ProcedureEntry[50] max
metadata: tags, urls, versions
[1]: ...
[99]: ...
categories: { 50+ defined categories }
$schema: "./schema/plugin-database.schema.json"
```

• **Appendix B: Estimated Database Metrics (100 Plugins)**

METRIC	VALUE
Total plugins	100
Total parameters	~8,500
Average params/plugin	85
Total factory presets	~4,500
Total procedures	~300
Categories	54
VST3 GUIDs	100
Raw JSON size	~5 MB
Gzipped JSON size	~600 KB
ajv validation time	< 50ms